Reto 6 – Integración Continua con Jenkins

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. Hasta ahora, se ha construido un sistema distribuido con:

- Servicios de negocio con persistencia y comunicación REST (Retos 1 y 2)
- Comunicación asincrónica basada en eventos (Reto 3)
- Seguridad centralizada con JWT y RBAC (Reto 4)
- Pruebas funcionales automatizadas con BDD (Reto 5)

A lo largo de los retos anteriores, cada equipo ha compilado, probado y desplegado sus microservicios de forma **manual**: ejecutando comandos de build localmente, corriendo pruebas en la terminal, y construyendo imágenes Docker a mano. Cuando un miembro del equipo introduce un cambio que rompe el build o las pruebas de otro servicio, el problema se descubre tarde — a veces solo cuando alguien ejecuta `docker-compose up` y el sistema falla.

En este reto se implementará un **pipeline de Integración Continua (CI)** utilizando **Jenkins**, de modo que cada cambio en el código sea automáticamente compilado, probado y empaquetado, detectando problemas **en minutos**, no en días.

Objetivo

Configurar un entorno de **Integración Continua** con **Jenkins** dentro del ecosistema Docker del proyecto. Cada microservicio debe contar con un **Jenkinsfile** que defina un pipeline automatizado con las etapas de **build**, **test**, **calidad** y **empaquetado Docker**, preparando al equipo para los requisitos CI/CD del proyecto final.

Introducción

¿Qué es la Integración Continua?

La **Integración Continua (CI – Continuous Integration)** es una práctica de desarrollo de software en la que los desarrolladores integran su código en un repositorio compartido de forma frecuente, y cada integración se **verifica automáticamente** mediante compilación y pruebas.

Aspecto	Sin CI	Con CI
Compilación	Manual, cuando alguien se acuerda	Automática, en cada push
Pruebas	Se ejecutan localmente (o no se ejecutan)	Se ejecutan automáticamente en cada cambio
Detección de errores	Al intentar desplegar (tarde)	En minutos tras el commit (temprano)
Confianza en el código	"En mi máquina funciona"	El pipeline lo verifica en un entorno limpio
Empaquetado	Manual: `docker build` en la terminal	Automático: imagen Docker generada por el pipeline

> **Principio fundamental:** Si duele hacerlo manualmente, automatícelo. Si lo automatiza, hágalo frecuentemente.

CI en el contexto de Microservicios

En una arquitectura de microservicios, la CI es especialmente valiosa porque:

1. **Múltiples servicios, múltiples lenguajes**: Cada servicio puede tener su propio stack tecnológico. Sin CI automatizada, es fácil que un servicio quede sin compilar o sin pruebas durante días.
2. **Deployments independientes**: Cada servicio debería poder compilarse, probarse y empaquetarse **de forma independiente**. Un `Jenkinsfile` por servicio garantiza esto.
3. **Contratos entre servicios**: Un cambio en un evento o en un endpoint puede romper otro servicio. Las pruebas automatizadas (unitarias e integración) en el pipeline detectan estas rupturas.

Jenkins

Jenkins es un servidor de automatización open-source, ampliamente utilizado para implementar pipelines de CI/CD.

Característica	Descripción
Open Source	Gratuito, con una gran comunidad y ecosistema de plugins
Pipeline as Code	Los pipelines se definen en un archivo `Jenkinsfile` versionado junto al código
Extensible	Miles de plugins: Docker, Git, reportes, notificaciones, etc.
Integración Docker	Puede construir imágenes Docker, ejecutar contenedores, y publicar en registries
Multi-lenguaje	No está atado a ningún lenguaje; ejecuta lo que usted configure

Pipeline Declarativo vs Scripted

Jenkins ofrece dos sintaxis para definir pipelines:

Aspecto	Declarativo	Scripted
Sintaxis	Estructurada con bloques `pipeline { }`	Groovy libre con `node { }`
Curva de aprendizaje	Menor, más legible	Mayor, más flexible
Recomendado para	La mayoría de casos	Escenarios muy complejos

Para este reto se utilizará el **pipeline declarativo** por su claridad y estructura:

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                // Compilar el proyecto
            }
        }
        stage('Test') {
            steps {
                // Ejecutar pruebas
            }
        }
        stage('Package') {
            steps {
                // Construir imagen Docker
            }
        }
    }

    post {
        always {
            // Acciones de limpieza o notificación
        }
    }
}
```

Conceptos clave del Jenkinsfile

Concepto	Descripción
`pipeline`	Bloque raíz que define todo el pipeline
`agent`	Dónde se ejecuta el pipeline (nodo, contenedor Docker, etc.)
`stages`	Conjunto de etapas secuenciales
`stage`	Una etapa con nombre (Build, Test, Package...)
`steps`	Los comandos a ejecutar dentro de una etapa
`post`	Acciones que se ejecutan después del pipeline (siempre, solo si falla, etc.)
`environment`	Variables de entorno disponibles en todo el pipeline

Concepto	Descripción
`tools`	Herramientas a configurar (Maven, JDK, Node, etc.)

Jenkins en Docker

Para que Jenkins funcione dentro del ecosistema Docker del proyecto, se necesita resolver un problema: **Jenkins necesita construir imágenes Docker desde dentro de un contenedor**.

Hay dos estrategias principales:

Estrategia	Cómo funciona	Ventajas	Desventajas
Docker Socket Mount	Se monta `/var/run/docker.sock` del host en el contenedor Jenkins	Sencillo, usa el Docker del host	Compartir el socket es un riesgo de seguridad (aceptable en entorno académico)
Docker-in-Docker (DinD)	Se ejecuta un daemon Docker dentro del contenedor Jenkins	Más aislado	Más complejo, posibles problemas con capas de almacenamiento

> **Recomendación para este reto**: Docker Socket Mount, por simplicidad.

Requisitos Generales

- Utilizar **Jenkins** como servidor de integración continua.
- Definir los pipelines en archivos **`Jenkinsfile`** versionados junto al código de cada microservicio.
- Usar la sintaxis de **pipeline declarativo**.
- Jenkins debe ejecutarse como un **servicio en Docker Compose**, accesible desde el navegador.
- Los pipelines deben poder ejecutarse **sin intervención manual** una vez configurados.

Retos

Los siguientes cinco puntos están organizados en orden de complejidad creciente. Se recomienda abordarlos en secuencia, ya que cada punto construye sobre las habilidades y la configuración del anterior.

Punto 1: Configuración de Jenkins en Docker Compose (1.0 pt)

Lo que se entrega

Un enfoque basado en un `Dockerfile` **personalizado** que pre-instala los plugins necesarios y el cliente Docker, de modo que la configuración sea **reproducible** y no dependa de pasos manuales en la interfaz de Jenkins.

`jenkins/Dockerfile`:

```
FROM jenkins/jenkins:lts-jdk17

# Instalar plugins necesarios de forma automatizada
# Referencia: https://github.com/jenkinsci/docker#preinstalling-plugins
RUN jenkins-plugin-cli --plugins \
    docker-pipeline \
    docker-workflow \
    pipeline-stage-view \
    workflow-aggregator \
    git \
    configuration-as-code \
    sonar

# Instalar cliente Docker dentro del contenedor Jenkins
USER root
RUN apt-get update && \
    apt-get install -y docker.io && \
    rm -rf /var/lib/apt/lists/*

# Agregar usuario jenkins al grupo docker para acceder al socket
RUN groupadd -f docker && usermod -aG docker jenkins

USER jenkins
```

`docker-compose.yml` (fragmento):

```yaml
services:
  jenkins:
    build:
      context: ./jenkins
      dockerfile: Dockerfile
    container_name: jenkins
    ports:
      - "9090:8080"       # Interfaz web
      - "50000:50000"     # Comunicación con agentes
    volumes:
      - jenkins_data:/var/jenkins_home
      - /var/run/docker.sock:/var/run/docker.sock  # Acceso al Docker del host
    environment:
      - JAVA_OPTS=-Djenkins.install.runSetupWizard=false
    networks:
      - microservices-network

volumes:
  jenkins_data:
```

> **Nota:** `jenkins-plugin-cli` es la herramienta oficial para instalar plugins desde la línea de comandos. Al incluirla en el Dockerfile, cada vez que se reconstruya la imagen, Jenkins tendrá los mismos plugins instalados sin intervención manual.

> **Nota:** `runSetupWizard=false` desactiva el asistente de configuración inicial, ya que los plugins se pre-instalan en el Dockerfile.

Lo que debe hacer

1. **Crear un directorio `jenkins/`** en la raíz del proyecto con un `Dockerfile` personalizado basado en la imagen oficial de Jenkins (`jenkins/jenkins:lts-jdk17` o `lts-jdk21`).
2. **Pre-instalar los plugins necesarios** en el Dockerfile usando `jenkins-plugin-cli` . Como mínimo:
   - `workflow-aggregator` (Pipeline)
   - `docker-pipeline` (Docker Pipeline)
   - `git` (integración Git)
   - Plugins adicionales según necesidad (JaCoCo, HTML Publisher, etc.)
3. **Instalar el cliente Docker** dentro del contenedor Jenkins para permitir la construcción de imágenes. Montar el socket Docker del host en el `docker-compose.yml` .
4. **Agregar Jenkins al `docker-compose.yml`** usando `build:` en lugar de `image:` , apuntando al Dockerfile personalizado. Asegúrese de que Jenkins esté en la misma red que los demás servicios.

5. **Levantar el sistema** y verificar que Jenkins es accesible desde `http://localhost:9090` . No debería requerir instalar plugins manualmente.
6. **Verificar la configuración** creando un pipeline de prueba que confirme que Docker funciona desde Jenkins:

```
pipeline {
    agent any
    stages {
        stage('Verificación') {
            steps {
                echo '✅ Jenkins está correctamente configurado'
                sh 'docker --version'
                sh 'docker ps'
            }
        }
    }
}
```

Si ambos comandos Docker se ejecutan correctamente, la configuración está completa.

Decisión técnica requerida

Usted debe decidir:

- ¿Qué plugins adicionales pre-instala en el Dockerfile según las herramientas que usará en los puntos posteriores (cobertura, análisis estático, reportes HTML)?
- ¿Monta el socket de Docker o utiliza DinD? ¿Cómo resuelve los permisos del socket Docker dentro del contenedor?
- ¿Desactiva el Setup Wizard (`runSetupWizard=false`) o lo mantiene para crear el usuario administrador manualmente?

Punto 2: Primer Jenkinsfile – Build y Test (1.0 pt)

Lo que se entrega

Ejemplo de un `Jenkinsfile` declarativo para un microservicio Java con Maven:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                // Si el Jenkinsfile está en el mismo repo, Jenkins hace checkout automáti
                // Si necesita clonar un repo diferente:
                // git url: 'https://github.com/usuario/mi-servicio.git', branch: 'main'
                echo 'Código fuente obtenido'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean compile -DskipTests'
            }
        }

        stage('Test') {
            steps {
                sh 'mvn verify'  // Ejecuta tests + genera reporte de cobertura (JaCoCo)
            }
        }
    }

    post {
        always {
            echo 'Pipeline finalizado'
        }
        success {
            echo '✅ Build y tests exitosos'
        }
        failure {
            echo '❌ El pipeline ha fallado'
        }
    }
}
```

La siguiente tabla muestra los comandos de build, test y generación de cobertura según el lenguaje:

Lenguaje	Comando Build	Comando Test + Cobertura
Java (Maven)	`mvn clean compile -DskipTests`	`mvn verify` (con JaCoCo configurado en `pom.xml`)
Java (Gradle)	`gradle build -x test`	`gradle test jacocoTestReport`
Python	`pip install -r requirements.txt`	`pytest --cov --cov-report=xml`
Node.js	`npm install`	`nyc --reporter=lcov npm test`
Go	`go build ./...`	`go test -coverprofile=coverage.out ./...`

> **Importante:** La generación de reportes de cobertura en la etapa de Test es esencial para que SonarQube (Punto 3) pueda importarlos y evaluar el Quality Gate. SonarQube **no ejecuta las pruebas**; solo lee los reportes generados aquí.

Lo que debe hacer

1. **Crear un `Jenkinsfile`** en la raíz del directorio de **uno** de sus microservicios. Elija el servicio que tenga pruebas unitarias implementadas.
2. **Definir las etapas** del pipeline:
   - **Checkout**: Obtener el código fuente
   - **Build**: Compilar el proyecto
   - **Test**: Ejecutar las pruebas unitarias **y generar el reporte de cobertura**
3. **Aprovisionar el pipeline como código** (sin configuración manual en la interfaz). El objetivo es que al levantar Jenkins con `docker-compose up`, los pipelines ya estén configurados. Hay varias estrategias:

   Opción A — Jenkins Configuration as Code (JCasC): Crear un archivo YAML que defina los jobs automáticamente. Este archivo se monta como volumen en el contenedor Jenkins:

```yaml
# jenkins/casc.yaml
jobs:
  - script: |
      pipelineJob('empleados-service') {
        definition {
          cpsScm {
            scm {
              git {
                remote {
                  url('https://github.com/usuario/empleados-service.git')
                }
                branches('*/main')
              }
            }
            scriptPath('Jenkinsfile')
          }
        }
      }
```

Y en el `docker-compose.yml`:

```yaml
services:
  jenkins:
    # ...
    volumes:
      - ./jenkins/casc.yaml:/var/jenkins_home/casc_configs/jobs.yaml
    environment:
      - CASC_JENKINS_CONFIG=/var/jenkins_home/casc_configs
```

Opción B — Script `init.groovy.d`: Colocar scripts Groovy en el directorio `init.groovy.d/` que se ejecutan al arrancar Jenkins. Estos scripts pueden crear los jobs programáticamente:

```groovy
// jenkins/init.groovy.d/create-jobs.groovy
import jenkins.model.*
import org.jenkinsci.plugins.workflow.job.WorkflowJob
import org.jenkinsci.plugins.workflow.cps.CpsScmFlowDefinition
import hudson.plugins.git.GitSCM

def jenkins = Jenkins.instance
// ... crear el job programáticamente
```

Opción C — Jenkinsfile local: Si el código de los microservicios está accesible desde el contenedor (por ejemplo, montado como volumen), configurar el pipeline para que lea el

`Jenkinsfile` directamente del sistema de archivos local.

> **Importante:** El plugin `configuration-as-code` debe estar pre-instalado en el Dockerfile de Jenkins (Punto 1) para que la Opción A funcione. El plugin `job-dsl` es necesario para definir jobs en JCasC.

4. **Configurar las herramientas necesarias** dentro del pipeline. Dado que Jenkins se ejecuta en un contenedor, puede necesitar:
   - Usar una imagen Docker como agente del pipeline (`agent { docker { image 'maven:3.9-eclipse-temurin-17' } }`)
   - O instalar las herramientas directamente en el contenedor Jenkins
5. **Verificar los resultados**: Los tests deben pasar y los reportes deben ser visibles en la interfaz de Jenkins.

Decisión técnica requerida

- **Aprovisionamiento de jobs**: ¿Qué estrategia usa para que los pipelines se configuren automáticamente al levantar Jenkins? ¿JCasC, init.groovy.d, o volúmenes locales? Justifique su elección.
- **Agente del pipeline**: ¿Ejecuta los pasos directamente en el contenedor Jenkins o utiliza contenedores Docker efímeros como agentes? La opción de contenedores efímeros es más limpia pero requiere la configuración Docker del Punto 1.

```
// Opción A: Ejecutar en Jenkins directamente
agent any

// Opción B: Usar contenedor Docker como agente
agent {
    docker {
        image 'maven:3.9-eclipse-temurin-17'
        args '-v /root/.m2:/root/.m2'  // Cache de dependencias
    }
}
```

- **Gestión de dependencias**: ¿Cómo evita que cada ejecución descargue todas las dependencias desde cero? (hint: volúmenes para cachés)

Punto 3: Calidad de Código con SonarQube (1.0 pt)

Lo que se entrega

SonarQube es una plataforma de inspección continua de código que permite evaluar la calidad del software de forma automatizada. A diferencia de herramientas individuales (Checkstyle, ESLint, flake8...) que solo cubren un lenguaje y un tipo de análisis, SonarQube ofrece una **solución unificada**:

Dimensión	Qué analiza
Bugs	Errores potenciales que pueden causar comportamiento inesperado
Vulnerabilidades	Problemas de seguridad (inyecciones SQL, XSS, etc.)
Code Smells	Código que funciona pero es difícil de mantener
Duplicación	Porcentaje de código duplicado
Cobertura	Porcentaje de código ejecutado por las pruebas
Deuda Técnica	Estimación del esfuerzo para corregir los problemas

¿Por qué SonarQube en este proyecto?

El proyecto final requiere **al menos 4 lenguajes de programación**. Con herramientas individuales, cada microservicio necesitaría su propia herramienta de análisis (Checkstyle para Java, ESLint para Node.js, flake8 para Python, golangci-lint para Go). SonarQube analiza **todos los lenguajes en una sola plataforma**, con un dashboard unificado y Quality Gates consistentes.

Quality Gate

Un **Quality Gate** es un conjunto de condiciones que el código debe cumplir para ser considerado aceptable. SonarQube incluye un Quality Gate por defecto ("Sonar way") que se puede personalizar:

Condición	Umbral por defecto
Cobertura en código nuevo	≥ 80%
Líneas duplicadas en código nuevo	≤ 3%
Bugs nuevos	0
Vulnerabilidades nuevas	0
Calificación de mantenibilidad	A

> Para este reto, se recomienda configurar un Quality Gate personalizado con **cobertura ≥ 70%** (alineado con el requisito del proyecto final).

SonarQube en Docker

SonarQube se dockeriza fácilmente, agregando dos servicios al Docker Compose:

```yaml
services:
  sonarqube:
    image: sonarqube:lts-community
    container_name: sonarqube
    ports:
      - "9000:9000"
    environment:
      - SONAR_JDBC_URL=jdbc:postgresql://sonar-db:5432/sonar
      - SONAR_JDBC_USERNAME=sonar
      - SONAR_JDBC_PASSWORD=sonar
    volumes:
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_logs:/opt/sonarqube/logs
      - sonarqube_extensions:/opt/sonarqube/extensions
    depends_on:
      - sonar-db
    networks:
      - microservices-network

  sonar-db:
    image: postgres:16-alpine
    container_name: sonar-db
    environment:
      - POSTGRES_DB=sonar
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar
    volumes:
      - sonar_db_data:/var/lib/postgresql/data
    networks:
      - microservices-network

volumes:
  sonarqube_data:
  sonarqube_logs:
  sonarqube_extensions:
  sonar_db_data:
```

Integración con el Pipeline

Cada lenguaje tiene su propia forma de enviar el análisis a SonarQube:

Lenguaje	Herramienta	Comando
Java (Maven)	Sonar Maven Plugin	`mvn sonar:sonar`
Java (Gradle)	Sonar Gradle Plugin	`gradle sonarqube`
Python	sonar-scanner	`sonar-scanner`
Node.js	sonar-scanner	`sonar-scanner`
Go	sonar-scanner	`sonar-scanner`

Para los lenguajes que no usan Maven/Gradle, se utiliza el **Sonar Scanner**, un cliente genérico que se configura con un archivo `sonar-project.properties` en la raíz del proyecto:

```
# sonar-project.properties
sonar.projectKey=mi-microservicio
sonar.projectName=Mi Microservicio
sonar.sources=src
sonar.host.url=http://sonarqube:9000
sonar.token=squ_xxxxxxxxxxxxxxxxxxxx
```

Ejemplo de la etapa SonarQube en el Jenkinsfile (Java/Maven):

```
stage('SonarQube') {
    steps {
        sh '''
            mvn sonar:sonar \
              -Dsonar.host.url=http://sonarqube:9000 \
              -Dsonar.token=${SONAR_TOKEN}
        '''
    }
}

stage('Quality Gate') {
    steps {
        timeout(time: 5, unit: 'MINUTES') {
            waitForQualityGate abortPipeline: true
        }
    }
}
```

> **Nota:** La etapa `waitForQualityGate` requiere el plugin **SonarQube Scanner for Jenkins** y un webhook configurado en SonarQube que notifique a Jenkins del resultado del análisis.

> **Importante:** Para que la cobertura aparezca en SonarQube, el microservicio debe generar un reporte de cobertura en un formato compatible (ej. JaCoCo XML para Java, `coverage.xml` para Python, `lcov.info` para Node.js). SonarQube **no ejecuta las pruebas**, solo **importa los reportes** generados en la etapa de Test.

Lo que debe hacer

1. **Agregar SonarQube y su base de datos PostgreSQL** al `docker-compose.yml`. Verificar que SonarQube es accesible desde `http://localhost:9000`.
2. **Configurar SonarQube**:
   - Acceder con las credenciales por defecto (`admin` / `admin`), cambiar la contraseña
   - Crear un **proyecto** para el microservicio del Punto 2
   - Generar un **token de autenticación** para el análisis desde Jenkins
   - Configurar un **Quality Gate** personalizado con cobertura ≥ 70%
3. **Configurar la generación de reportes de cobertura** en el microservicio. El reporte debe generarse durante la etapa de Test (Punto 2). Por ejemplo:
   - **Java**: JaCoCo genera `target/site/jacoco/jacoco.xml` automáticamente con `mvn test`
   - **Python**: `pytest --cov --cov-report=xml` genera `coverage.xml`
   - **Node.js**: `nyc --reporter=lcov npm test` genera `coverage/lcov.info`
   - **Go**: `go test -coverprofile=coverage.out ./...`

4. **Agregar la etapa de SonarQube** al Jenkinsfile:
    - Ejecutar el scanner/plugin que envía el código y los reportes de cobertura a SonarQube
    - Esperar el resultado del **Quality Gate**
    - Si el Quality Gate **no pasa**, el pipeline debe **fallar**
5. **Verificar en SonarQube** que el dashboard del proyecto muestra:
    - Métricas de bugs, vulnerabilidades y code smells
    - Porcentaje de cobertura
    - Porcentaje de duplicación
    - Estado del Quality Gate (Passed/Failed)
6. **Probar el Quality Gate**:
    - Temporalmente reduzca las pruebas para que la cobertura caiga por debajo del 70%
    - O introduzca un bug intencional que SonarQube detecte
    - Verifique que el Quality Gate falla y el pipeline se detiene

Decisión técnica requerida

- ¿Cómo gestiona el token de SonarQube en el pipeline? ¿Lo almacena en Jenkins Credentials, en variables de entorno del Docker Compose, o en un archivo?
- ¿Configura el Quality Gate por defecto ("Sonar way") o crea uno personalizado? ¿Con qué umbrales?
- ¿Cómo configura el webhook de SonarQube a Jenkins para que `waitForQualityGate` funcione?
- Para servicios en lenguajes diferentes a Java, ¿cómo instala y ejecuta `sonar-scanner`? ¿Lo incluye en el `Dockerfile` del servicio, lo descarga en el pipeline, o usa una imagen Docker con el scanner pre-instalado?

Punto 4: Empaquetado Docker y Pruebas E2E (1.0 pt)

Lo que se entrega

Este punto cierra el ciclo del pipeline integrando dos etapas finales: el **empaquetado** del microservicio como imagen Docker y la ejecución de **pruebas de extremo a extremo (E2E)** contra el sistema completo desplegado.

Empaquetado Docker

Ejemplo de la etapa de empaquetado Docker en el Jenkinsfile:

```
stage('Package') {
    steps {
        script {
            def imageName = "mi-microservicio"
            def imageTag = "${env.BUILD_NUMBER}"

            sh "docker build -t ${imageName}:${imageTag} -t ${imageName}:latest ."

            echo "✅ Imagen construida: ${imageName}:${imageTag}"
        }
    }
}
```

Y cómo configurar un **Docker Registry local** en el Docker Compose:

```yaml
services:
  registry:
    image: registry:2
    container_name: docker-registry
    ports:
      - "5000:5000"
    volumes:
      - registry_data:/var/lib/registry
    networks:
      - microservices-network

volumes:
  registry_data:
```

Pruebas E2E con BDD

En el **Reto 5** se construyó una suite de pruebas funcionales automatizadas con BDD (Cucumber/Behave). Esa suite se ejecutó manualmente después de levantar el sistema. Ahora, esa misma suite se incorpora al pipeline CI para que se ejecute **automáticamente** como última etapa de verificación.

El flujo es:

```
Package → Deploy (docker-compose up) → E2E Tests (BDD) → Cleanup (docker-compose down)
```

Ejemplo de la etapa E2E en el Jenkinsfile:

```
stage('E2E Tests') {
    steps {
        // Levantar el sistema completo con las imágenes recién construidas
        sh 'docker-compose -f docker-compose.yml up -d --build'

        // Esperar a que los servicios estén listos (health checks)
        sh '''
            echo "Esperando a que los servicios estén listos..."
            sleep 30  // O implementar un script de espera con health checks
        '''

        // Ejecutar la suite BDD
        sh '''
            cd e2e-tests
            mvn test                    # Java/Cucumber
            # behave                    # Python/Behave
            # npx cucumber-js           # Node.js/Cucumber.js
        '''
    }
    post {
        always {
            // Limpiar: apagar el sistema desplegado
            sh 'docker-compose -f docker-compose.yml down'
        }
    }
}
```

> **Nota:** La etapa E2E puede ser costosa en tiempo. En un pipeline real, se puede configurar para ejecutarse solo en ciertos eventos (ej. merge a `main`), no en cada commit. Para este reto, se ejecutará siempre.

Lo que debe hacer

1. **Agregar la etapa `Package`** al Jenkinsfile del microservicio del Punto 2:
   - Construir la imagen Docker del microservicio
   - Etiquetar la imagen con el número de build y con `latest`
   - Verificar que la imagen se construyó correctamente
2. **(Opcional pero recomendado)** Agregar un **Docker Registry local** al Docker Compose:
   - Publicar la imagen construida al registry local
   - Verificar que la imagen es accesible desde el registry

```
stage('Publish') {
    steps {
        script {
            def imageName = "localhost:5000/mi-microservicio"
            def imageTag = "${env.BUILD_NUMBER}"

            sh "docker tag mi-microservicio:${imageTag} ${imageName}:${imageTag}"
            sh "docker push ${imageName}:${imageTag}"

            echo "📤 Imagen publicada en registry: ${imageName}:${imageTag}"
        }
    }
}
```

3. **Agregar la etapa de pruebas E2E** al pipeline:
   - Levantar el sistema completo con Docker Compose
   - Esperar a que todos los servicios estén operativos (health checks o espera con reintentos)
   - Ejecutar la **suite BDD del Reto 5** contra el sistema desplegado
   - Apagar el sistema al finalizar (en el bloque `post { always { } }`)
4. **Extender a un segundo microservicio**: Crear un `Jenkinsfile` para otro microservicio del sistema, **en un lenguaje de programación diferente**. Esto demuestra que el pipeline CI es adaptable a cualquier stack tecnológico.
5. **Verificar el pipeline completo**: Ejecutar ambos pipelines y confirmar que todas las etapas (Checkout → Build → Test → SonarQube → Quality Gate → Package → E2E) pasan exitosamente.

Decisión técnica requerida

- **Naming convention**: ¿Qué convención de nombres usa para las imágenes Docker? ¿Incluye prefijo de proyecto, versionado semántico, hash del commit?
- **Multi-stage builds**: ¿Sus Dockerfiles utilizan multi-stage builds para optimizar el tamaño de las imágenes? ¿Cómo se integra esto con el pipeline?
- **Registry**: ¿Local o DockerHub? ¿Cómo configura la autenticación si usa un registry externo?
- **Health check antes de E2E**: ¿Cómo verifica que todos los servicios están listos antes de ejecutar las pruebas BDD? ¿Usa un script de espera, polling a los endpoints de health check, o `depends_on` con condiciones?
- **Aislamiento de datos**: ¿Cómo garantiza que las pruebas E2E no interfieran con datos de ejecuciones anteriores? ¿Usa `docker-compose down -v` para limpiar volúmenes?

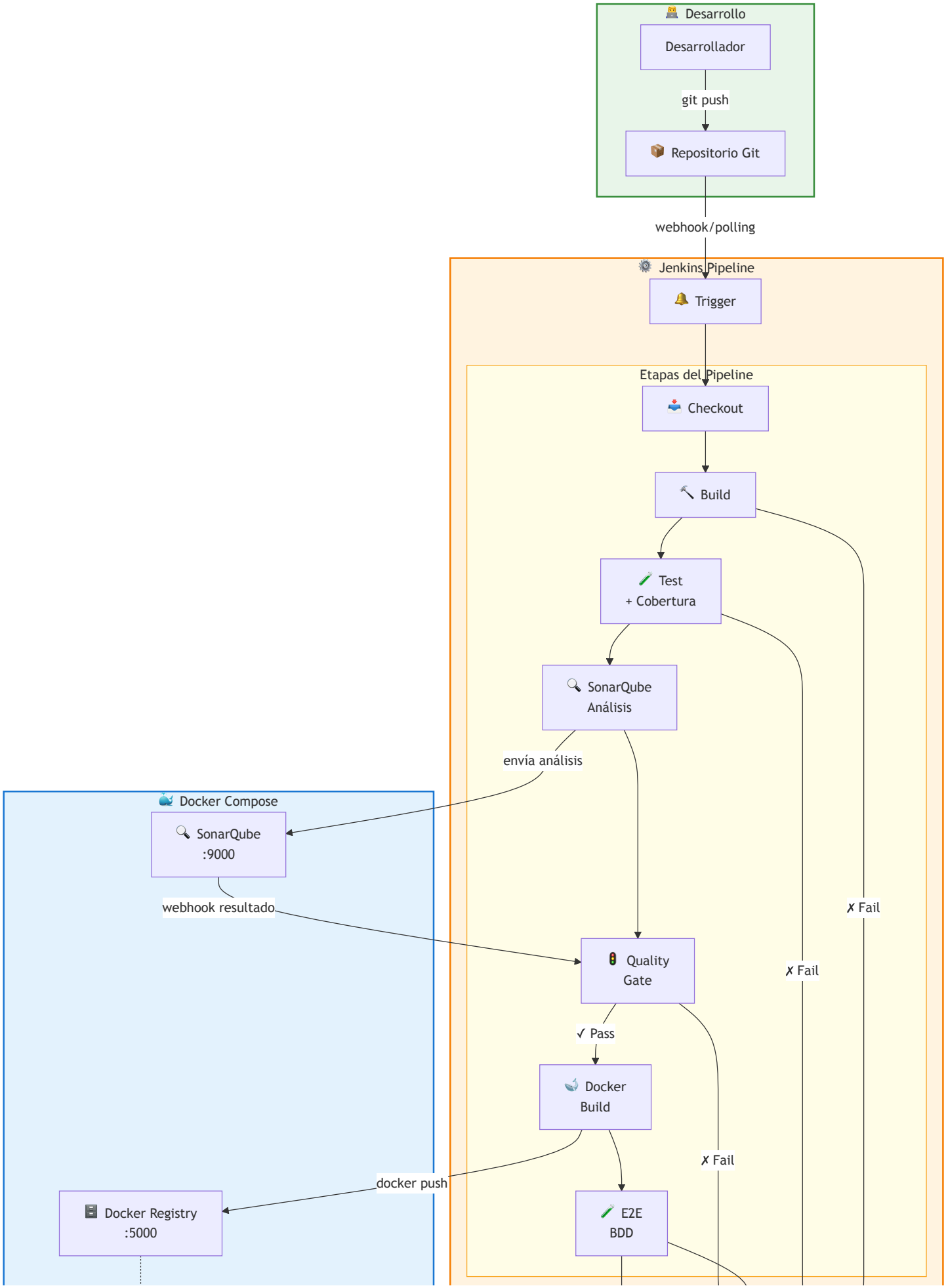
Punto 5: Reproducibilidad y Documentación (1.0 pt)

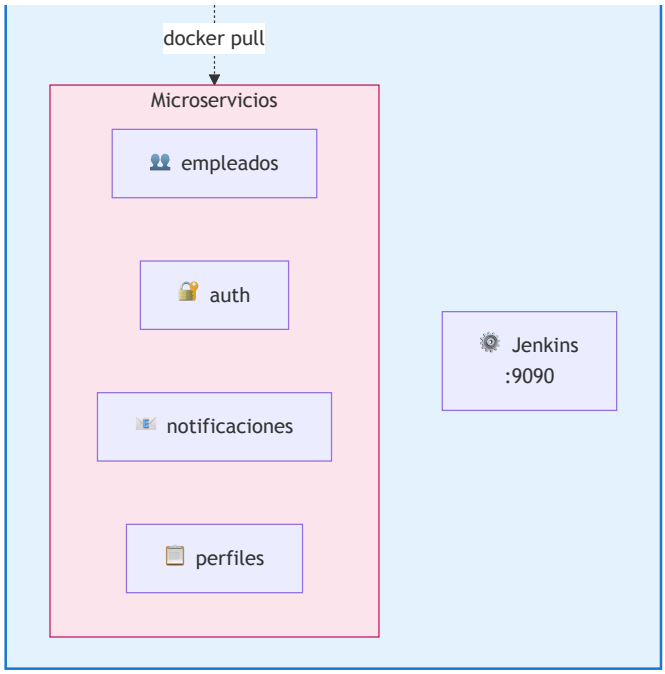
Lo que debe hacer

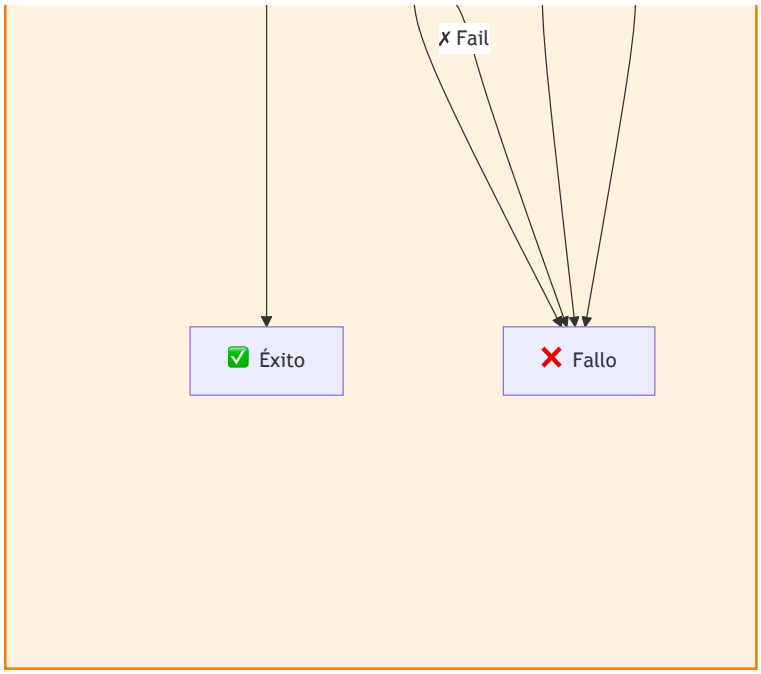
1. **Ejecución consistente del pipeline**: Ejecute cada pipeline al menos **3 veces** consecutivas y confirme que todos los builds pasan de forma consistente. Si hay builds intermitentes (*flaky builds*), identifique y corrija la causa.
2. **Simular un fallo**: Modifique intencionalmente el código para que:
   - Una prueba unitaria falle → el pipeline debe detenerse en la etapa **Test** con mensaje descriptivo
   - La cobertura caiga por debajo del 70% → el Quality Gate de SonarQube debe fallar y el pipeline detenerse en la etapa **Quality Gate**
   - El Dockerfile tenga un error → el pipeline debe detenerse en la etapa **Package**
   - Un escenario BDD falle → el pipeline debe detenerse en la etapa **E2E Tests**

   Verifique que en cada caso el pipeline falla **en la etapa correcta** y el log indica claramente **qué** falló.
3. **Documentar en el README.md:**
   - Breve explicación de qué es CI y por qué se integra en el proyecto
   - **Acceso a Jenkins**: URL, credenciales por defecto
   - **Instrucciones de configuración**:
     a. Cómo levantar el sistema con Jenkins incluido
     b. Cómo obtener la contraseña inicial de Jenkins
     c. Cómo crear/importar los pipelines
     d. Cómo ejecutar un pipeline manualmente
   - **Descripción de las etapas** del pipeline y qué verifica cada una
   - **Cómo interpretar los resultados**: qué significa cada etapa en verde/rojo
   - Capturas de pantalla de un pipeline exitoso en Jenkins
4. **(Opcional)** Configurar un **webhook** para que el pipeline se dispare automáticamente al hacer push. Esto puede lograrse con:
   - **Gitea/Gogs** como servidor Git local dentro del Docker Compose
   - **Jenkins Generic Webhook Trigger Plugin**
   - Polling SCM como alternativa simple (configurable en el Jenkinsfile con `triggers { pollSCM('H/5 * * * *') }`)

Diagrama de Arquitectura CI


Desarrollo
Desarrollador
git push
Repositorio Git
webhook/polling
Jenkins Pipeline
Trigger
Etapas del Pipeline
Checkout
Build
Test
+ Cobertura
SonarQube
Análisis
envía análisis
Docker Compose
SonarQube
:9000
webhook resultado
Quality
Gate
✗ Fail
✗ Fail
✓ Pass
Docker
Build
✗ Fail
docker push
Docker Registry
:5000
E2E
BDD





Entregables

- `docker-compose.yml` actualizado con Jenkins, SonarQube (y opcionalmente Docker Registry) como servicios.
- Un `Jenkinsfile` por microservicio (mínimo 2, en lenguajes diferentes).
- `Dockerfile` personalizado de Jenkins (si aplica).
- `README.md` actualizado según lo especificado en el Punto 5.
- Código versionado en GitHub.

Consideraciones

- Jenkins debe ejecutarse como **servicio del Docker Compose**, no como instalación local. El sistema completo debe levantarse con `docker-compose up --build`.
- Los `Jenkinsfile` deben estar **versionados junto al código** de cada microservicio, no configurados solo en la interfaz de Jenkins.
- Las credenciales y configuraciones sensibles deben manejarse mediante **variables de entorno** o **Jenkins Credentials**, no hardcodeadas en el Jenkinsfile.
- Si Jenkins necesita herramientas específicas (Maven, Node, Python, Go), se recomienda usar **imágenes Docker como agentes** del pipeline para evitar instalar todo en el contenedor Jenkins.
- El pipeline debe ser **idempotente**: ejecutar el mismo pipeline múltiples veces debe producir el mismo resultado.

Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, correspondiendo **1 punto** a cada uno de los retos:

#	Reto	Valor	Aspectos a evaluar
1	**Configuración de Jenkins**	1.0	Jenkins accesible como servicio Docker. Dockerfile personalizado con plugins pre-instalados. Acceso a Docker configurado. Pipeline de verificación funcional. Configuración reproducible sin pasos manuales.
2	**Jenkinsfile – Build y Test**	1.0	Jenkinsfile declarativo con etapas Checkout, Build y Test. Generación de reportes de cobertura configurada. Pipelines aprovisionados como código (JCasC, init.groovy.d, o volúmenes). Pruebas unitarias pasando. Agente del pipeline correctamente configurado.
3	**Calidad con SonarQube**	1.0	SonarQube dockerizado y accesible. Proyecto configurado con análisis de calidad y cobertura. Quality Gate personalizado con cobertura ≥70%. Pipeline falla si el Quality Gate no pasa. Dashboard de SonarQube con métricas visibles.
4	**Empaquetado Docker y Pruebas E2E**	1.0	Etapa Package construye imagen Docker. Etapa E2E levanta el sistema y ejecuta la suite BDD del Reto 5 automáticamente. Al menos 2 Jenkinsfiles (2 microservicios, 2 lenguajes). Cleanup del sistema después de E2E.
5	**Reproducibilidad y documentación**	1.0	Pipelines consistentes en múltiples ejecuciones. Fallos se reportan en la etapa correcta. README completo con instrucciones, acceso a Jenkins, y capturas de un pipeline exitoso.

> **Nota:** Se valorará especialmente que los pipelines sean **claros y mantenibles**. Un buen `Jenkinsfile` debe poder ser leído y entendido por cualquier miembro del equipo, actuando como documentación ejecutable del proceso de integración del servicio.